

VoicelQ — Speech & NLP Intelligence Platform

Project Documentation

1. Project Overview

Project Name

VoicelQ — Speech & NLP Intelligence Platform

Project Idea

VoicelQ is an AI-powered speech processing and sentiment analysis system that records or uploads audio, processes the signal using Digital Signal Processing (DSP), extracts acoustic features, converts speech into text using Whisper AI, and finally analyzes the sentiment of the text using multiple NLP models.

The project combines:

- Audio Processing
- Digital Signal Processing (DSP)
- Speech Recognition
- Feature Extraction
- Natural Language Processing (NLP)
- Machine Learning
- Deep Learning
- Interactive GUI Development

2. Main Objectives

The system aims to:

1. Record or upload audio signals.
2. Clean and preprocess speech signals.

- 3. Extract meaningful acoustic features.
- 4. Convert speech into English text.
- 5. Detect the sentiment of the spoken text.
- 6. Compare multiple NLP models.
- 7. Provide an interactive user interface.
- 8. Visualize audio analysis and DSP operations.

3. Technologies Used

Technology	Purpose
Python	Main programming language
Streamlit	Web application GUI
CustomTkinter	Desktop GUI
Librosa	Audio processing and feature extraction
Faster-Whisper	Speech recognition and translation
Transformers	BERT sentiment analysis
Scikit-learn	ML models (Naive Bayes & SVM)
Matplotlib	Data visualization
NLTK	NLP preprocessing
Noisereduce	Noise reduction
NumPy	Numerical operations
SoundDevice	Audio recording
SoundFile	Audio saving/loading
gTTS	Text-to-Speech

4. System Architecture

Complete Pipeline

Audio Input



Signal Acquisition

↓

Digital Signal Processing (DSP)

↓

Voice Activity Detection (VAD)

↓

Noise Reduction

↓

Feature Extraction

↓

Whisper Speech Recognition

↓

Translation to English

↓

Text Cleaning

↓

Sentiment Analysis

↓

Final Result Visualization

.

5. Project Components

Part A — Audio Acquisition & Analysis

Purpose

This stage captures the audio signal either from:

- Microphone recording

- Uploaded audio file

Input Types

Supported formats:

- WAV
- MP3
- FLAC
- OGG

Recording Settings

sample_rate = 16000

Why 16000 Hz?

16000 Hz is commonly used in speech applications because:

- Human speech mainly exists below 8 kHz.
- It reduces computational cost.
- It works efficiently with Whisper and DSP algorithms.

Part B — Audio Visualization

1. Waveform Visualization

The waveform displays:

- Amplitude over time
- Loudness changes
- Silence vs speech regions

Purpose

- Understand audio behavior
- Detect noise
- Observe speech activity

Library Used

`librosa.display.waveshow()`

2. FFT — Fast Fourier Transform

FFT converts audio from:

Time Domain → Frequency Domain

Purpose

- Analyze frequencies inside the signal
- Detect dominant frequency components
- Understand spectral behavior

Formula

$$X(k) = \sum x(n)e^{-j2\pi kn/N}$$

Benefits

- Useful for speech analysis
- Helps detect pitch and harmonics
- Important in DSP systems

Part C — Digital Signal Processing (DSP)

1. Signal Normalization

Purpose

Normalize audio amplitude to:

- Prevent clipping
- Standardize volume
- Improve model performance

Equation

normalized = signal / max(abs(signal))

2. Pre-Emphasis Filter

Purpose

Amplify high frequencies because speech naturally loses high-frequency energy.

Formula

$$y[n] = x[n] - \alpha x[n-1]$$

Where:

- $\alpha = 0.97$

Benefits

- Improves speech clarity
- Enhances feature extraction
- Makes MFCC extraction better

3. Voice Activity Detection (VAD)

Purpose

Remove silent regions from audio.

Why Important?

- Reduces processing time
- Removes unnecessary frames
- Improves speech recognition accuracy

Method Used

librosa.effects.split()

4. Noise Reduction

Purpose

Remove background noise from speech.

Library Used

noisereduce

Benefits

- Cleaner audio
- Better transcription
- Better NLP accuracy

5. Framing

Purpose

Split speech into small overlapping windows.

Settings

- Frame Length = 25 ms
- Frame Stride = 10 ms

Why Framing?

Speech is non-stationary globally but approximately stationary over short durations.

Window Used

Hamming Window

Benefits

- Smooth transitions
- Reduces spectral leakage
- Better FFT and MFCC extraction

Part D — Feature Extraction

Feature extraction converts audio into numerical representations that machine learning models can understand.

1. MFCC — Mel Frequency Cepstral Coefficients

Purpose

Extract speech characteristics based on human hearing.

Number of Features

`n_mfcc = 13`

Why MFCC?

MFCC is one of the most important speech features because it:

- Mimics human auditory perception
- Captures speech timbre
- Works efficiently in speech recognition

Pipeline

Audio → FFT → Mel Filter Bank → Log → DCT → MFCC

Benefits

- Compact representation
- High speech recognition accuracy
- Robust for NLP pipelines

2. Mel Spectrogram

Purpose

Represent frequency energy over time using the Mel scale.

Why Mel Scale?

The Mel scale represents frequencies similar to how humans hear sound.

Benefits

- Better speech visualization
- Useful for deep learning
- Captures temporal frequency changes

3. Spectral Centroid

Purpose

Measure the “center of mass” of frequencies.

Interpretation

- High centroid → brighter sound
- Low centroid → deeper sound

4. Spectral Bandwidth

Purpose

Measure frequency spread around the centroid.

Benefits

- Helps characterize speech texture
- Useful in audio classification

Part E — Speech Recognition

Model Used

Faster-Whisper

Why Faster-Whisper?

- Faster inference
- Lower memory usage

- Accurate multilingual speech recognition
- CPU optimized

Whisper Tasks

1. Transcription

Convert speech to text.

2. Translation

Translate speech from multiple languages into English.

Supported Languages

The project supports:

- Arabic
- English
- French
- German
- Russian
- Spanish
- Italian
- Chinese
- Japanese
- Turkish
- Auto Detection

Whisper Parameters

Parameter	Purpose
-----------	---------

model size = small	Faster inference
beam_size = 2	Better decoding accuracy
vad_filter = True	Remove silence

Output Generated

The system produces:

- Transcribed text
- Detected language
- Confidence score
- Inference time

Part F — NLP Sentiment Analysis

The project uses multiple sentiment analysis models.

1. Naive Bayes

Type

Probabilistic Machine Learning Model

Input

TF-IDF vectors

Advantages

- Fast
- Lightweight
- Good baseline performance

Disadvantages

- Assumes feature independence
- Lower contextual understanding

2. SVM — Support Vector Machine

Type

Supervised Machine Learning Model

Purpose

Classify text using decision boundaries.

Advantages

- Strong performance on text classification
- Effective with sparse vectors

Disadvantages

- Slower than Naive Bayes
- Requires feature engineering

3. BERT / DistilBERT

Type

Transformer-based Deep Learning Model

Why BERT?

BERT understands:

- Context
- Grammar
- Word relationships
- Sentence meaning

Advantages

- Very high accuracy
- Context-aware predictions
- Better semantic understanding

Why DistilBERT?

- Smaller model
- Faster inference
- Reduced memory usage

6. Text Preprocessing

Before feeding text into ML models:

Operations Applied

1. Remove HTML Tags

<text> → removed

2. Remove URLs

http://...

3. Remove Mentions & Hashtags

@user #topic

4. Remove Numbers

5. Remove Emojis

6. Remove Special Characters

7. Lowercasing

8. Space Normalization

7. User Interface

GUI Types

1. Streamlit Web Application

Features:

- Modern interface

- Interactive visualizations
- Real-time processing
- Theme switching
- Tabs organization
- Sidebar settings

2. CustomTkinter Desktop App

Features:

- Voice recording
- Text input
- Sentiment display
- Lightweight design

8. Streamlit Interface Sections

1. Record & Analyse

Contains:

- Microphone recording
- File upload
- Automatic pipeline execution
- Sentiment results

2. DSP Clean

Displays:

- Noise reduction
- Cleaned signals

- DSP visualizations

3. Features

Displays:

- MFCC
- Spectrograms
- Frequency analysis
- Acoustic metrics

9. Data Visualization

The project visualizes:

Audio Visualizations

- Waveform
- FFT Spectrum
- Mel Spectrogram
- MFCC Heatmaps

NLP Visualizations

- Sentiment confidence
- Model comparison
- Dataset analysis

EDA Visualizations

- Sentiment distribution
- Text length histogram
- Top words frequency

10. Dataset Analysis (EDA)

Dataset Used

Training dataset containing:

- Positive samples
- Negative samples

EDA Goals

- Understand data distribution
- Analyze text lengths
- Find common words
- Detect dataset balance

11. Model Comparison

Model	Speed	Accuracy	Complexity
Naive Bayes	Very Fast	Medium	Low
SVM	Fast	High	Medium
BERT	Slower	Very High	High

12. Project Workflow

Complete Workflow

Step 1 — Audio Input

Record or upload audio.

Step 2 — DSP Processing

Apply:

- Normalization
- Pre-emphasis

- VAD
- Noise reduction
- Framing

Step 3 — Feature Extraction

Extract:

- MFCC
- Mel Spectrogram
- Spectral Features

Step 4 — Speech Recognition

Use Whisper to:

- Detect language
- Transcribe speech
- Translate to English

Step 5 — NLP Analysis

Run:

- Naive Bayes
- SVM
- BERT

Step 6 — Final Visualization

Display:

- Sentiment result
- Confidence
- Graphs
- Audio metrics

13. Key AI Concepts Used

1. DSP — Digital Signal Processing

Manipulating audio signals mathematically.

2. NLP — Natural Language Processing

Allowing machines to understand human language.

3. ASR — Automatic Speech Recognition

Converting speech into text.

4. Transformer Models

Deep learning architectures that understand contextual language relationships.

5. Feature Engineering

Transforming raw data into meaningful numerical representations.

14. Challenges Faced

1. Noise in Audio

Solved using:

- Noise reduction
- VAD
- Pre-emphasis

2. Multilingual Speech

Solved using:

- Whisper translation
- Auto language detection

3. Model Performance

Solved using:

- Cached models
- Optimized inference
- Faster-Whisper

4. UI Responsiveness

Solved using:

- Streamlit caching
- Efficient processing
- Threaded recording

15. Performance Optimizations

Techniques Used

Streamlit Caching

@st.cache_resource

@st.cache_data

Benefits

- Faster loading
- Reduced memory usage

- Better responsiveness

Faster-Whisper Optimization

`compute_type="int8"`

Benefits

- Lower RAM usage
- Faster CPU inference

16. Security & Error Handling

The project includes:

- Try/except blocks
- File existence checking
- Missing model handling
- Upload validation
- Runtime error prevention

17. Future Improvements

Possible future upgrades:

1. Real-Time Streaming

Continuous live speech analysis.

2. Emotion Detection

Detect:

- Happiness
- Anger

- Fear
- Sadness

3. Speaker Identification

Recognize different speakers.

4. Cloud Deployment

Deploy using:

- AWS
- Azure
- Hugging Face Spaces

5. Mobile Application

Convert system into Android/iOS app.

6. Advanced Deep Learning

Use:

- Wav2Vec2
- Whisper Large
- LLM integration

18. Conclusion

VoicelQ is a complete AI pipeline that integrates:

- Audio Processing
- DSP
- Feature Extraction
- Speech Recognition
- Machine Learning
- Deep Learning

- [NLP](#)
- [GUI Development](#)

The project demonstrates how multiple AI domains can work together inside one intelligent system capable of analyzing speech and understanding human emotions from audio.

19. References

Libraries & Frameworks

- [Streamlit Documentation](#)
- [Librosa Documentation](#)
- [Hugging Face Transformers](#)
- [Faster-Whisper](#)
- [Scikit-learn Documentation](#)
- [NLTK Documentation](#)
- [Matplotlib Documentation](#)